

Computer Architecture: *Core and ALU*

Dr. Colin O'Flynn

Lectures

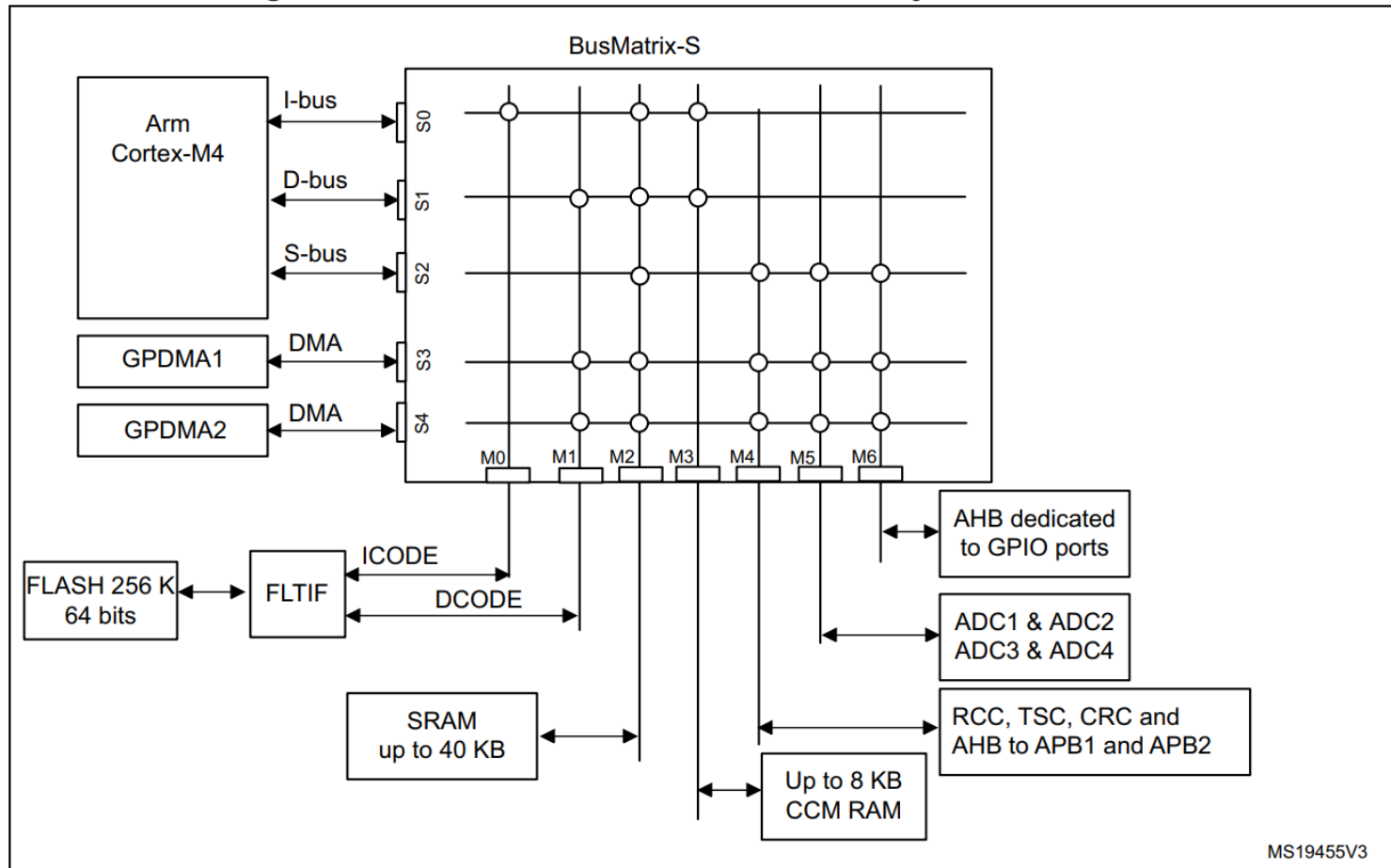
- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store, access, and control data?
- **How does the machine execute instructions?**
- *How is execution made more efficient?*
- *How do we measure execution?*

Tie to ISA Lecture

- Several of the topics “should” be call-backs to the ISA lecture
- In many instances we are going to look in more detail about *why* choices were made for the ISA here.

What's hidden in here?

Figure 1. STM32F303xB/C and STM32F358xC system architecture



- We now go inside the magic.

Arm vs RISC-V



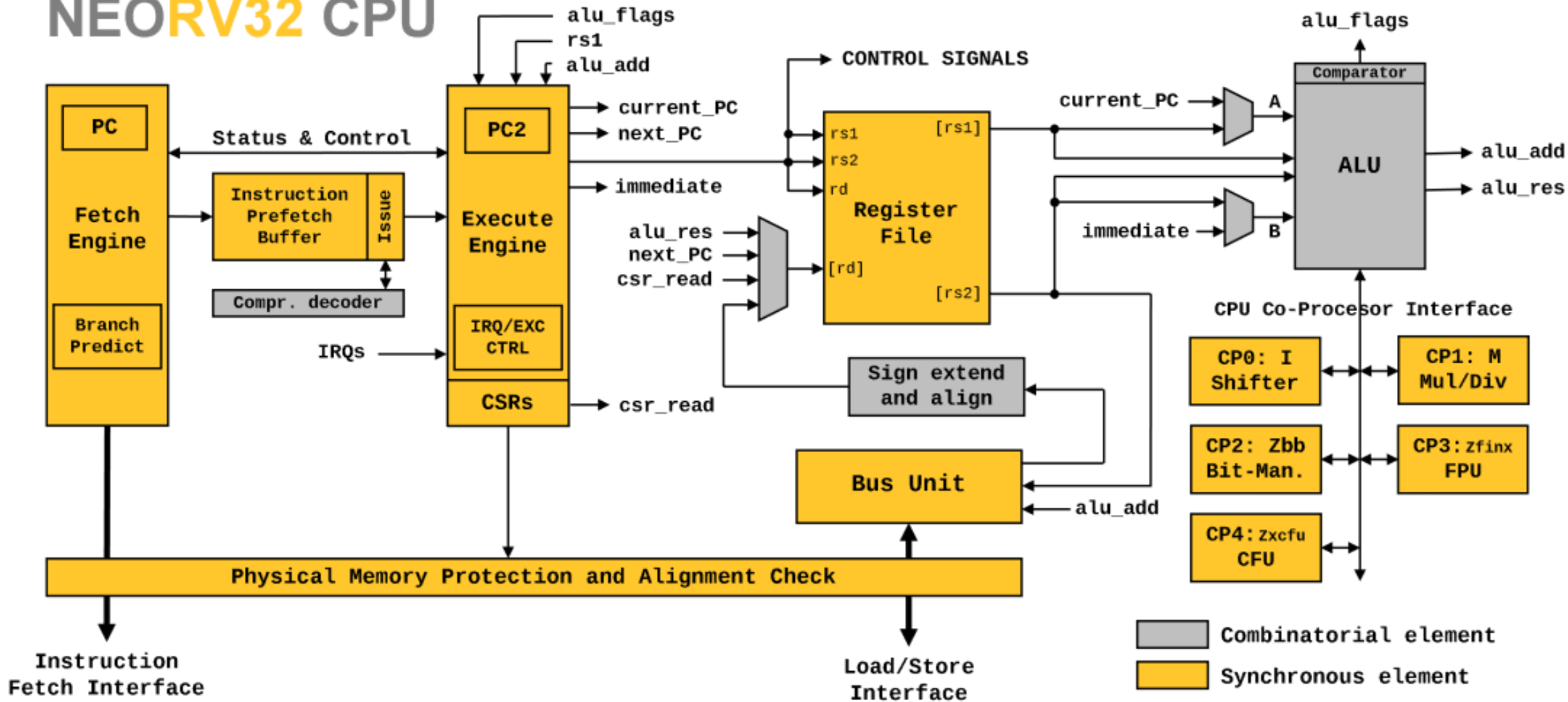
- RISC-V is unique as the Instruction Set Architecture is *Open Source*.
 - Other device (such as Arm) have a company behind them – you cannot use the *instruction set* without paying them \$\$\$.
- There are still commercial (closed source) *implementations* of RISC-V.
- But there are also LOTS of open-source processors, simulators, etc.

We can inspect the design of these open-source processors.

Arm vs RISC-V

- In the following slides I'll switch between RISC-V & Arm cores.
- They are very similar for these purposes
- Most code examples will use Arm assembly

NEORV32 CPU



Instruction Lifecycle

1. Fetch instruction
2. Decode instruction
3. Read registers
4. Execute operation
5. Access memory (if needed)
6. Write result

E.g., **ADD R1 , R2 , R3**

1. Fetch E0821003
2. Decode ADD R1, R2, R3
3. Read registers
 Operand 1 <= R2
 Operand 2 <= R3
4. Execute
 Result = R2 + R3
 Update flags
5. Access memory (not needed)
6. Writeback
 R1 <= Result

Fetch

- Reads data from memory.
- May include *prefetch* buffer of possible instructions (e.g., assume we read next address)
- May include *issue engine* to deal with compressed instructions

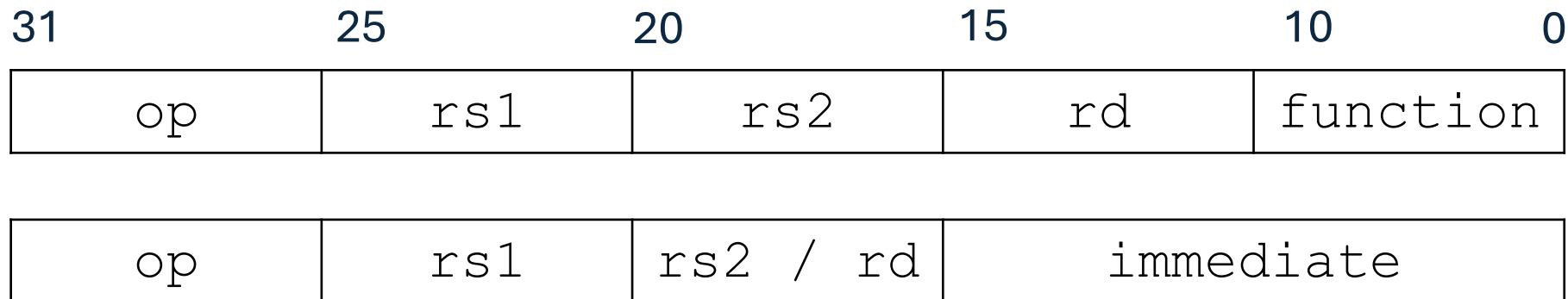
https://github.com/stnolting/neorv32/blob/main/rtl/core/neorv32_cpu_frontend.vhd

Decoding

- Example of such decoding was in the 2.0 ISA Slides:

`Instruction Rd, Rs, Operand2`

- Given 32-bits to encode our instructions, we may invent two simple instruction encoding formats for our processor, e.g.:



Decode

- Decoding is normally a simple mapping process of bits to various subsystems.
- There is a fine line between fetch and decode – typically before any action is taken we consider this the *Frontend*
- Actual work is done in the *Backend*:

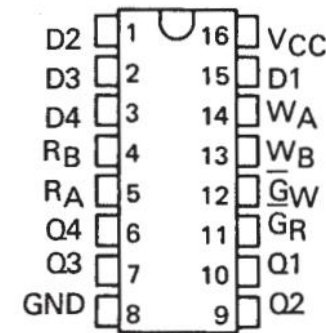
https://github.com/stnolting/neorv32/blob/main/rtl/core/neorv32_cpu_control.vhd

Register File

- Separate Read/Write Addressing Permits Simultaneous Reading and Writing
- Fast Access Times . . . Typically 20 ns
- Organized as 4 Words of 4 Bits
- Expandable to 512 Words of n-Bits
- For Use as:
 - Scratch-Pad Memory
 - Buffer Storage between Processors
 - Bit Storage in Fast Multiplication Designs
- 3-State Outputs
- SN54LS170 and SN74LS170 Are Similar But Have Open Collector Outputs

SN54LS670 . . . J OR W PACKAGE
SN74LS670 . . . D OR N PACKAGE

(TOP VIEW)

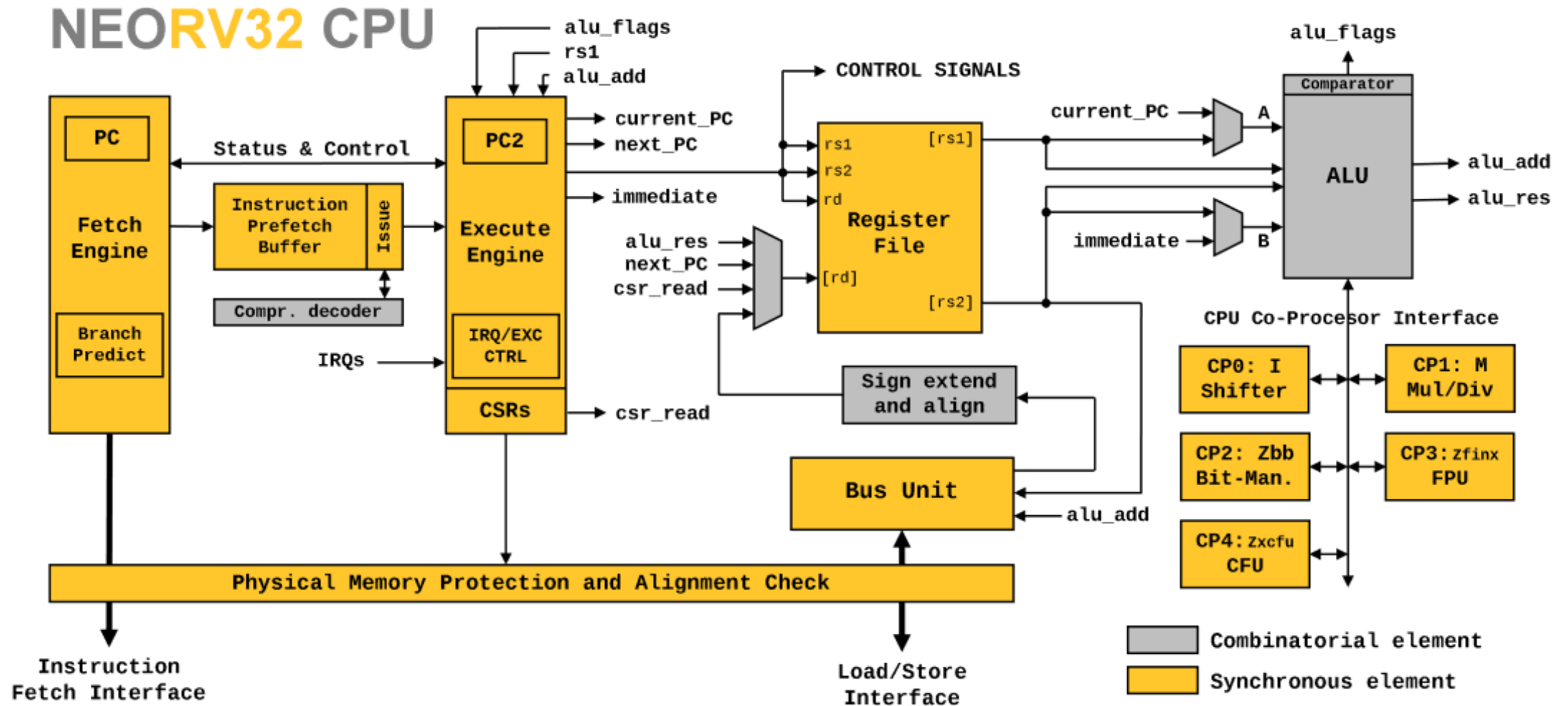


SN54LS670 . . . FK PACKAGE

(TOP VIEW)

- Registers are very fast memory near CPU
- Discussed in ISA lectures

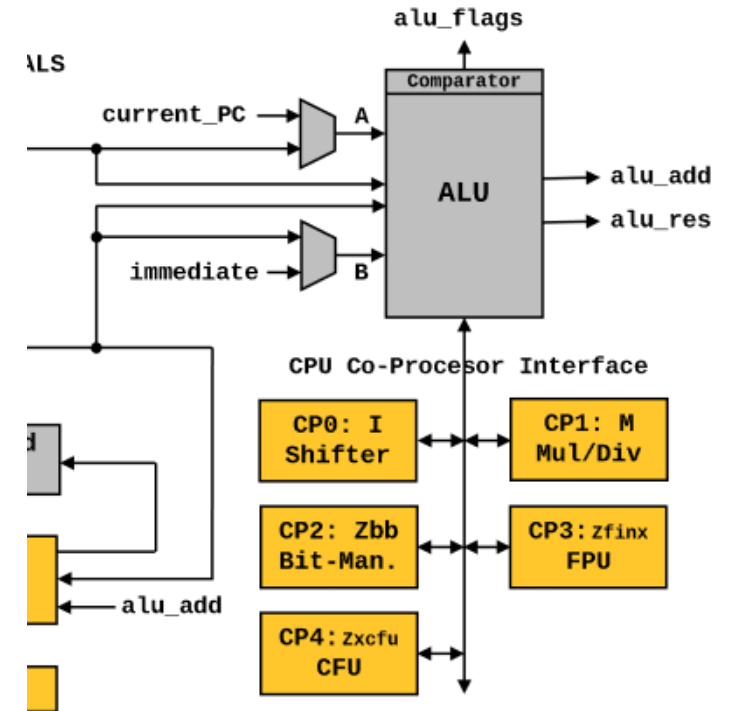
Read Registers



Execute

- This is the *backend* – control (decode) and routing of data to appropriate resource.
- Basic operations are actually very simple:

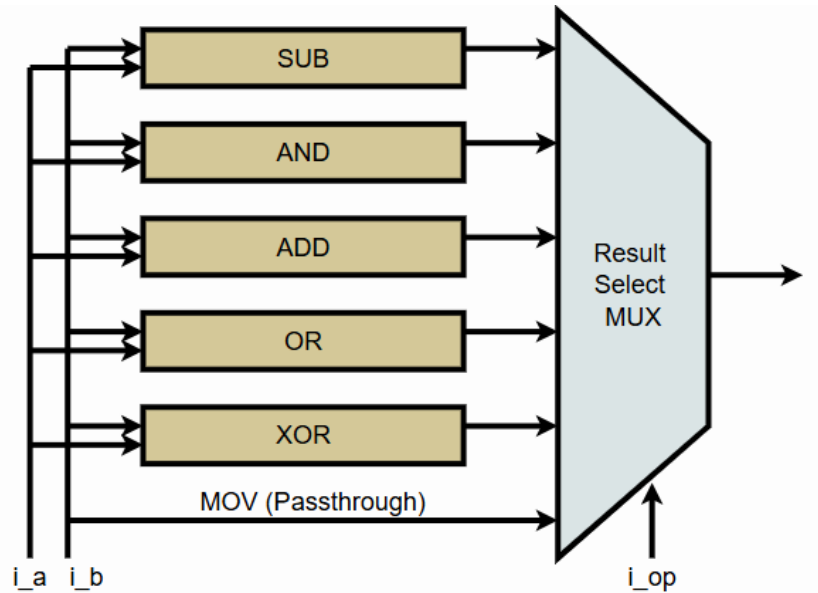
```
-- ALU Core -----  
-----  
alu_core: process(ctrl_i, addsub, cp_res, opb, rs1_i)  
begin  
  res_o <= (others => '0');  
  case ctrl_i.alu_op is  
    when alu_op_zero_c => res_o <= (others => '0');  
    when alu_op_add_c  => res_o <= addsub(31 downto 0);  
    when alu_op_cp_c   => res_o <= cp_res;  
    when alu_op_slt_c  => res_o(0) <= addsub(addsub'left); -- carry/borrow  
    when alu_op_movb_c => res_o <= opb;  
    when alu_op_xor_c  => res_o <= opb xor rs1_i;  
    when alu_op_or_c   => res_o <= opb or  rs1_i;  
    when alu_op_and_c  => res_o <= opb and rs1_i;  
    when others       => res_o <= (others => '0');  
  end case;  
end process alu_core;
```



https://github.com/stnolting/neorv32/blob/main/rtl/core/neorv32_cpu_alu.vhd

ALU

- Combination of simple arithmetic operations & flags
- Performs comparisons, arithmetic, bitwise operations



<https://zipcpu.com/zipcpu/2017/08/11/simple-alu.html>

Status Registers

- N = Negative result from ALU
- Z = Zero result from ALU
- C = ALU operation carry out
- V = ALU operation oVerflowed

Carry Flag

- Carry flag is normally set as a result of arithmetic operation:
 - SUB
 - ADD
- Instructions can take the carry as an **input**:
 - ADC (Add with Carry)
 - SBC (Subtract With Carry)

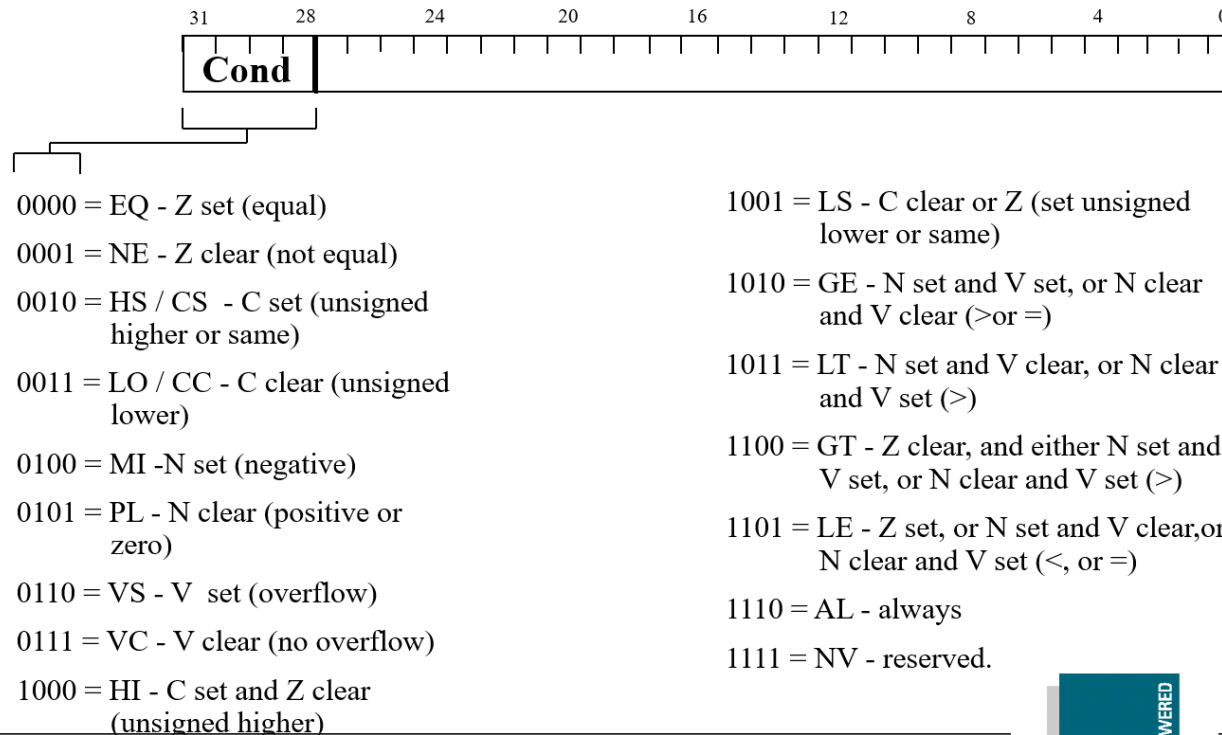
Carry Flag

- In Arm specifically:
 - On addition, carry flag is set if result $> 2^{32}$ (e.g., overflows unsigned 32-bit)
 - On subtraction, carry flag is cleared if a borrow occurs
 - *This is opposite some architectures, where carry flag is also a borrow flag (set to 1)*
- Carry flag also used for compare instructions
 - CMP = SUB but without a destination (only sets flags)
 - E.g:
CMP R0, #42 => SUB R0, #42

if R0 == 42:
 Zero flag set (can compare if equal)
if R0 < 42:
 Carry flag clear (borrow occurred)
If R0 >= 42:
 Carry flag set (no borrow)

Arm: Control Flow Reminder

The Condition Field



Overflow Flag

- Overflow flag can be thought of “signed carry”
 - Remember – sign is using 2’s complement format, where upper bit indicates sign
- Set when result of signed operation has opposite sign

e.g., 0x7FFFFFFF = Largest 32-bit signed number possible

$0x7FFFFFFF + 0x7FFFFFFF = 0xFFFFFFFF$

- Input have sign bit clear
- Result does not set carry flag, but does set overflow flag

Comparisons to 50 years(!) ago

Am2901B/Am2901C

Four-Bit Bipolar Microprocessor Slice

Am2901B/Am2901C

DISTINCTIVE CHARACTERISTICS

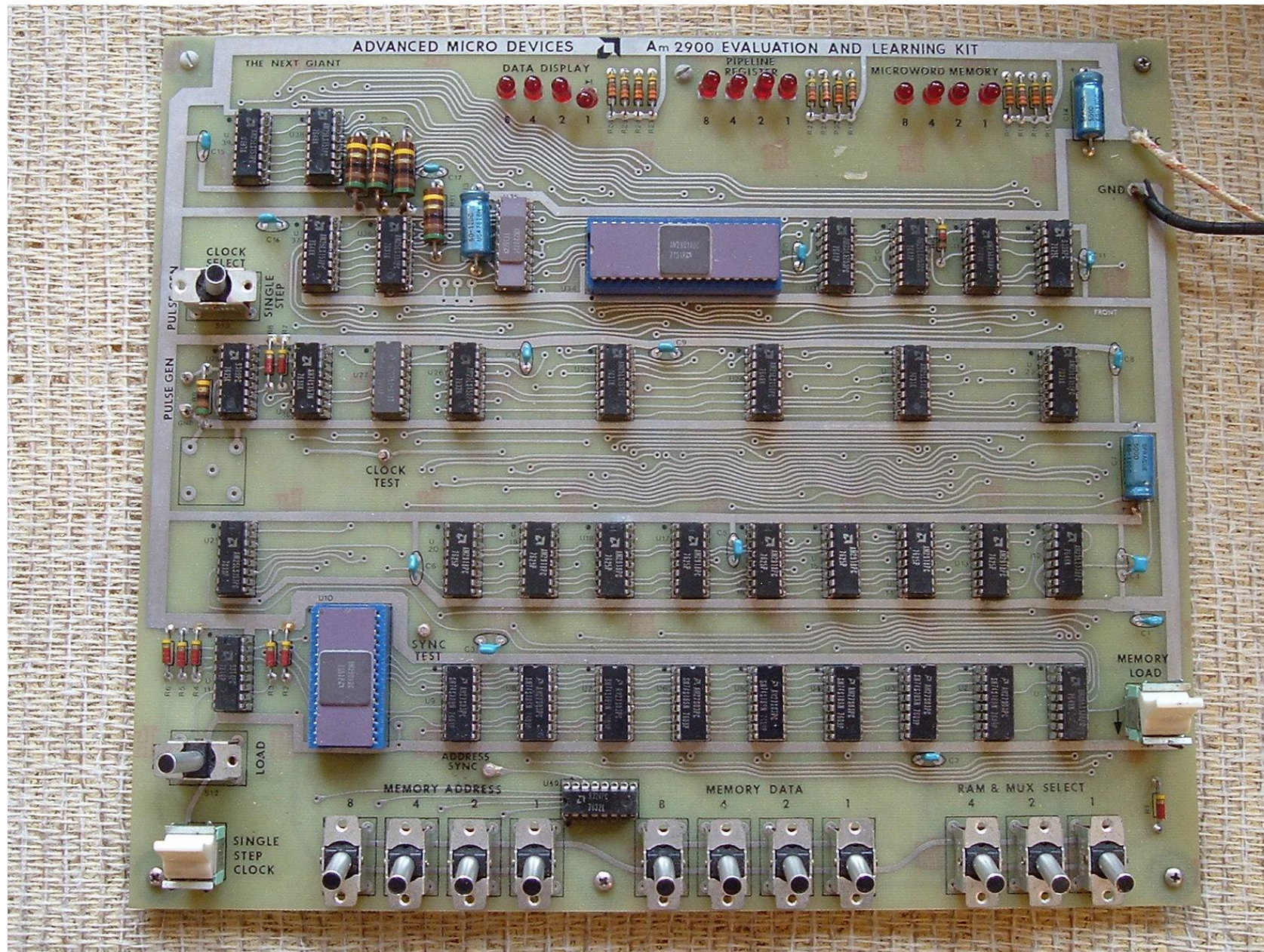
- **Two-address architecture** – Independent simultaneous access to two working registers saves machine cycles.
- **Eight-function ALU** – Performs addition, two subtraction operations, and five logic functions on two source operands.
- **Expandable** – Connect any number of Am2901s together for longer word lengths.
- **Left/right shift independent of ALU** – Add and shift operations take only one cycle.
- **Four status flags** – Carry, overflow, zero, and negative.
- **Flexible data source selection** – ALU data is selected from five source ports for a total of 203 source operand pairs for every ALU function.

GENERAL DESCRIPTION

The Am2901 industry standard four-bit microprocessor slice is a high-speed cascadable ALU intended for use in CPUs, peripheral controllers, and programmable microprocessors. The microinstruction flexibility of the Am2901

groups of three bits each and selects the ALU source operands, the ALU function, and the ALU destination register. The microprocessor is cascadable with full look ahead or with ripple carry, has three-state outputs, and

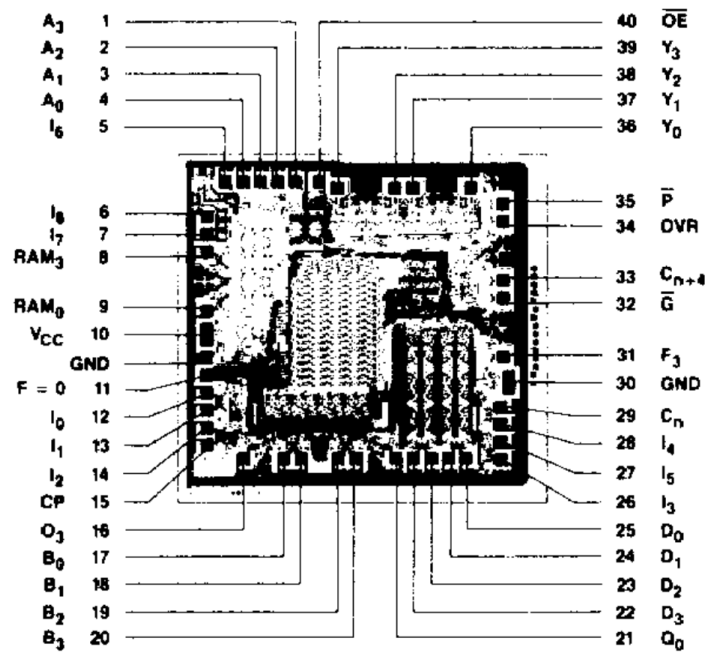
5



RELATED PRODUCTS

Part No.	Description
Am2902	Carry Look-Ahead Generator
Am2904	Status and Shift Control Unit
Am2910	Microprogram Controller
Am2914	Vectored Priority Interrupt Controller
Am2917	Bus Transceiver
Am2918	Pipeline Register
Am2920	Octal Register
Am2922	Condition Code MUX
Am2925	System Clock Generator
Am2940	DMA Address Generator
Am2952	Bidirectional I/O Port
Am27S35	Registered PROM

METALLIZATION AND PAD LAYOUT



Mnemonic	MICRO CODE				ALU SOURCE OPERANDS	
	I ₂	I ₁	I ₀	Octal Code	R	S
AQ	L	L	L	0	A	Q
AB	L	L	H	1	A	B
ZQ	L	H	L	2	0	Q
ZB	L	H	H	3	0	B
ZA	H	L	L	4	0	A
DA	H	L	H	5	D	A
DQ	H	H	L	6	D	Q
DZ	H	H	H	7	D	0

Figure 3. ALU Source Operand Control.

Mnemonic	MICRO CODE				ALU Function	SYMBOL
	I ₅	I ₄	I ₃	Octal Code		
ADD	L	L	L	0	R Plus S	R + S
SUBR	L	L	H	1	S Minus R	S - R
SUBS	L	H	L	2	R Minus S	R - S
OR	L	H	H	3	R OR S	R v S
AND	H	L	L	4	R AND S	R ^ S
NOTRS	H	L	H	5	$\bar{R}$ AND S	$\bar{R} \wedge S$
EXOR	H	H	L	6	R EX-OR S	R v S
EXNOR	H	H	H	7	R EX-NOR S	$\overline{R \vee S}$

Figure 4. ALU Function Control.

e.g., Simple Increment with AM2901

EXERCISE NUMBER: 3 NAME: AM2901 ARITHMETIC OPERATIONS

MICROPROGRAM MEMORY

RAM & MUX	7				6				5			4			3			2				1				0				NOTES				
	BRANCH ADDRESS				NEXT INSTRUCTION CONTROL				MUX1	DEST. SELECT			MUX0	SOURCE SELECT			C11	ALU			A				B				D				NEXT ADDRESS CONTROL	DATA CONTROL
	BR3	BR2	BR1	BR0	P3	P2	P1	P0		I8	I7	I6		I2	I1	I0		I5	I4	I3	A3	A2	A1	A0	B3	B2	B1	B0	D3	D2	D1	D0		
0										0	1	1		0	1	1	1	0	0	0					0	0	0	0						Inc R0
1										0	1	1		0	1	1	0	0	0	1					0	0	0	0						Dec R0
2										0	1	1		1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	R0 + 3
3																																		

Memory Access

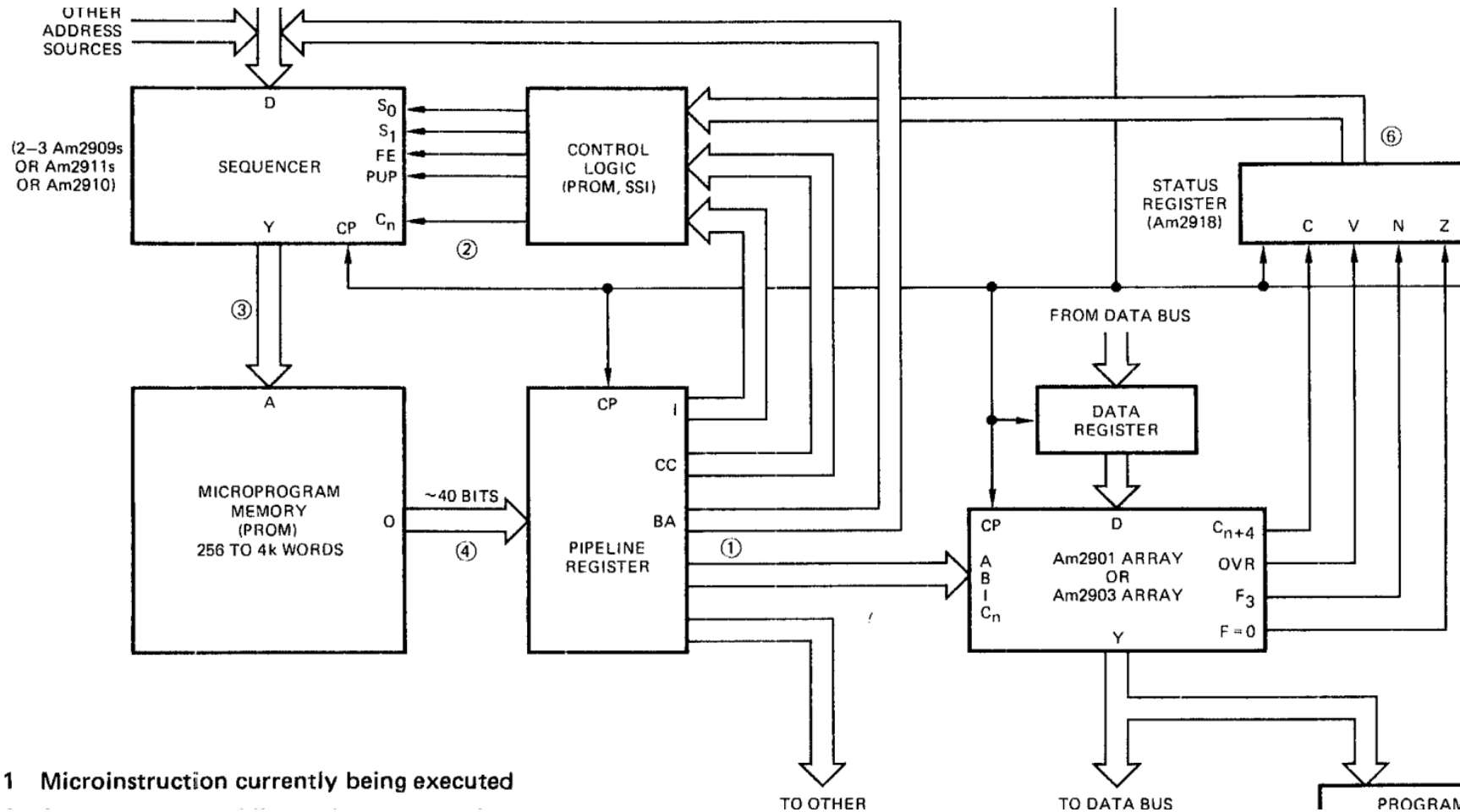
- If memory access is “special” (such as load-store architectures) this may be a separate cycle
- Perform any load/store to the memory system, *not just register access*

Write Back

- Output registers are updated

(This is often part of another stage)

Microprogram Instructions



1 Microinstruction currently being executed

- Even our 50 year old CPU had the idea of *microcode* or *microprogram*
- Instructions implemented from ALU operations

Superscalar Processors

- Before multi-core processors were popular, we had the idea of superscalar
- Allows execution of simultaneous instructions – often duplicating some resources (ALU)

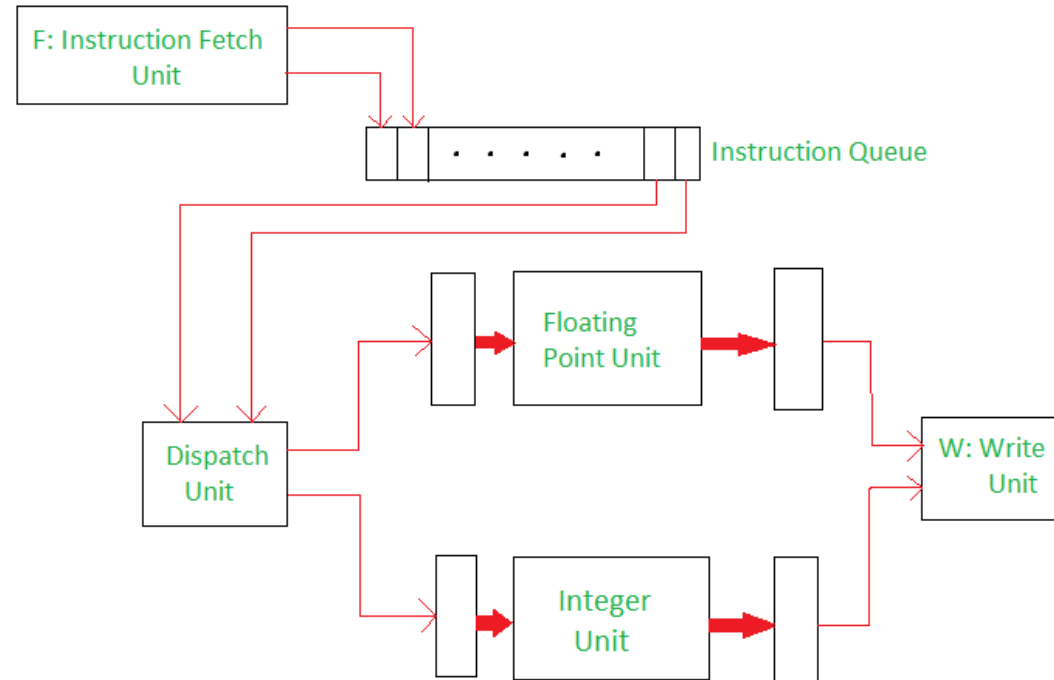
```
ldr r0, #35
```

```
add r2, r3, r4
```

```
add r0, r3
```

```
add r3, r5
```

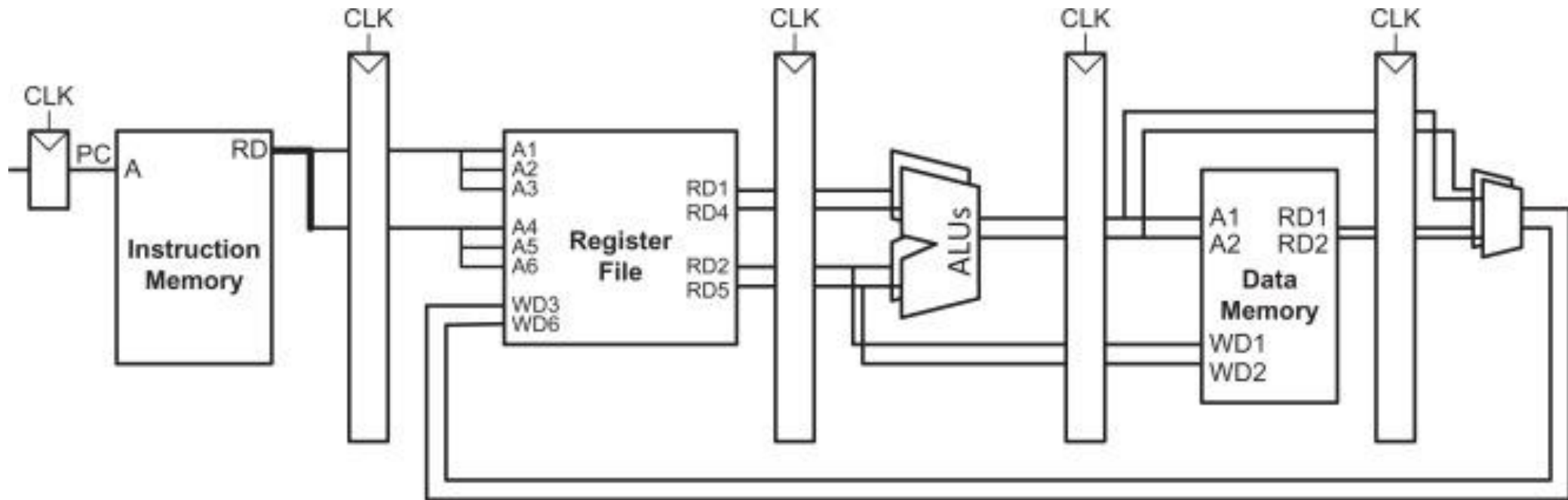
Superscalar: Different Execution Units (ALU)



Processor with Two Execution Units

<https://www.geeksforgeeks.org/computer-organization-architecture/superscalar-architecture/>

Superscalar: Multiple of Same Execution Unit (ALU)



Instruction & ALU Summary

- Instructions are executed in several stages, some of these may be “combined”:
 1. Fetch instruction
 2. Decode instruction
 3. Read registers
 4. Execute operation
 5. Access memory (if needed)
 6. Writeback
- Status register flags are an import part of the ALU usage
- Microcode-based instructions are a common feature (that may be “hidden” from you)